

Aula 1 – Introdução a C++

Prof. Sandino Jardim

UFMT – Campus Araguaia

Retirados do Plano de Ensino da disciplina

Justificativa

- Esta disciplina visa familiarizar o aluno com conceitos e práticas de **programação orientada a objetos** usando a linguagem de programação **C++**.

Objetivo Geral

- Capacitar o aluno a desenvolver software utilizando o **paradigma de orientação a objetos**.

Objetivos Específicos

- Estudo dos princípios da **programação orientada a objetos**;
- Estudo da linguagem de programação **C++**;
- Uso de ferramentas para o desenvolvimento de software.

Comparando

Programação Orientada a Objetos (POO)

- Paradigma de Programação
- Baseada nos conceitos de objetos e classes

C++

- Linguagem de Programação de Propósito Geral
- Suporta diferentes paradigmas de programação
 - Procedural
 - Abstrata
 - Genérica
 - Orientada a objetos

Breve história

- Criado por Bjarne Stroustrup nos laboratórios da AT&T Bell em Nova Jersey, no início dos anos 1980
- Combinou orientação a objetos de Simula67 com a elegância e simplicidade de C
- Originalmente chamado de “C with Classes”



Principal evolução



C++ é um superconjunto de C

Tudo o que você sabe atualmente sobre C se aplica a C++



Adições mais importantes em relação a C são:

Classes, herança, sobrecarga de função e de operadores, polimorfismo



Novas capacidades permitem melhores programas

clareza, extensibilidade, facilidade de manutenção

Aplicações de C++

- Linguagem versátil para lidar com programas de qualquer dimensão
- Adequado para praticamente qualquer projeto:
 - Editores, compiladores, banco de dados, sistemas de comunicação, etc.
- Permite aliar a capacidade de abstração de objetos do mundo real oferecido por OO com o detalhamento a nível de máquina de C
- Muito utilizado para softwares de infraestruturas e componentes de recursos limitados
- Sistemas Operacionais, e-commerce, jogos, navegadores, [etc.](#)

Compilador e IDE



Use a que achar melhor,
mas certifique-se de
instalar o compilador



Recomendações:

Codeblocks
Dev C++
NetBeans
Eclipse
VSC

Hello, World!

Primeiro programa em C++

```
#include <iostream>
int main()
{
    std::cout << "Hello, World!\n";
}
```

Biblioteca padrão de
entrada e saída

Operador indicando o
sentido do fluxo

Fluxo padrão de saída

Compilação: `g++ -o hw helloworld.cpp`

Tipos primitivos

Modificadores Comuns

Esses termos podem ser usados com tipos inteiros para alterar o intervalo de valores:

- unsigned: Apenas números não negativos (dobra o valor máximo).
- signed: (Padrão) Permite números positivos e negativos.

Categoria	Tipo (C++)	Descrição	Exemplo de Uso
Inteiros	int	Números inteiros (positivos e negativos).	int idade = 30;
	short	Inteiros menores, economiza memória.	short dias = 365;
	long	Inteiros maiores.	long populacao = 100000;
	char	Usado para armazenar caracteres (tratado como inteiro pequeno).	char inicial = 'A';
Ponto Flutuante	float	Números com casas decimais (precisão simples).	float preco = 19.99f;
	double	Números com casas decimais (precisão dupla). Recomendado.	double media = 7.5;
Booleano	bool	Armazena valores lógicos: true (verdadeiro) ou false (falso).	bool ativo = true;

Categoria	Operadores	Descrição	Exemplo
Aritméticos	+, -, *, /	Adição, Subtração, Multiplicação e Divisão.	x = a + b;
	%	Módulo (Resto da divisão inteira).	r = 10 % 3; → r=1
	++, --	Incremento e Decremento (Adiciona ou subtrai 1).	x++;
Relacionais	==, !=	Igualdade e Diferença.	if (a == 5)
	>, <, >=, <=	Maior que, Menor que, Maior ou Igual, Menor ou Igual.	if (a > b)
Lógicos	&&	AND lógico (Ambos devem ser verdadeiros).	if (a > 0 && b < 10)
		OR lógico	if (a > 0 b < 10)
	!	NOT lógico (Inverte o valor booleano).	if (!ativo)
Atribuição	=	Atribui o valor da direita à variável da esquerda.	x = 10;
	+=, -=, *= etc.	Atribuição composta (ex: x += 5; é o mesmo que x = x + 5;).	total += valor;

Funções e *namespace*

Example2.cpp

```
#include <iostream>

using namespace std;

double quadrado(double x)
{
    return x*x;
}

void imprime_quadrado(double x)
{
    cout << "o quadrado de " << x << " eh " << square(x) << "\n";
}

int main()
{
    imprime_quadrado(3.0);
}
```

Comando de entrada

Example3.cpp

```
#include <iostream>

using namespace std;

int main() {
    int a, b;

    cout << "Input A\n";
    cin >> a;

    cout << "Input B\n";
    cin >> b;

    cout << a << " + " << b << " = " << a+b;

}
```

Média aritmética

```
#include <iostream>

using namespace std;

double media(double n1, double n2) {
    return (n1+n2)/2.0;
}

int main() {
    double n1, n2;

    cout << "Digite valor 1\n";
    cin >> n1;

    cout << "Digite valor 2\n";
    cin >> n2;

    cout << "O valor da media eh: " << media(n1,n2) << "\n";

}
```

Laços de repetição

```
for (int i=0; i<10; i++)  
    v2[i]=v1[i];
```

```
int v[] = {0,1,2,3,4,5,6,7,8,9};  
  
for (auto x : v) // for each x in v  
    cout << x << '\n';
```

```
#include <iostream>

int main() {
    // 1. Declaração e inicialização dos vetores
    const int TAMANHO = 10;
    int v1[TAMANHO] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int v2[TAMANHO]; // 0 vetor de destino v2 é declarado, mas não
    inicializado

    // Copia o vetor 1 (v1) para o vetor 2 (v2)
    for (int i = 0; i < TAMANHO; i++) {
        v2[i] = v1[i];
    }

    // 2. Imprimir o vetor copiado (v2) para verificar
    std::cout << "Elementos de v2 (copia de v1):\n";
    for (int i = 0; i < TAMANHO; i++) {
        std::cout << v2[i] << "\n";
    }

    return 0;
}
```

Laço de repetição range-based (for-each)

```
#include <iostream>

int main() {
    // Declaração e inicialização do vetor
    int v[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    // Laço de repetição range-based (for-each)
    std::cout << "Elementos do vetor v:\n";
    for (auto x : v) {
        // Exibe cada elemento seguido de uma quebra de linha
        std::cout << x << "\n";
    }

    return 0;
}
```

Exercícios de Fixação

- **MDC (Máximo Divisor Comum)**

- Crie a função `calcular_mdc(A, B)` para retornar o MDC entre dois inteiros A e B utilizando o Algoritmo de Euclides.

- **Número Primo**

- Crie a função `eh_primo(N)` que retorna `true` se o inteiro N for primo, e `false` caso contrário.

- **Soma dos Dígitos**

- Leia um inteiro positivo N e calcule a soma de todos os seus dígitos.
- *Ex:* Se $N=345$, a soma é 12 ($3+4+5$).

- **Série Harmônica**

- Leia um inteiro N. Calcule a soma parcial da Série Harmônica ($H\{N\}$):
- $H\{N\} = 1 + 1/2 + \dots + 1/N$

- **Potência Iterativa**

- Crie a função `calcular_potencia(B, E)` para retornar B^E **sem** usar a função `pow()`.